

# Geometrinin Ötesi: Yazılım Mimarisinde Dinamik, Kuvvet, Niyet ve Anlam Katmanları

Ömer Yavuz

May 2026

## Giriş

Kod geometrisi — modüllerin düğüm, bağımlılıkların kenar, dosya hacminin kütle olarak modellendiği yapısal analiz — sistemin belirli bir andaki durağan resmini ortaya koymaktadır. Ancak bir mimariyi yeterince anlamak için yapısal görüntünün ötesine geçilmesi gerekmektedir: aynı geometrinin zaman içinde nasıl değiştiği, bu değişimi hangi kuvvetlerin ürettiği, sistemin hangi niyete hizmet ettiği ve son tahlilde hangi insan deneyimini mümkün kıldığı soruları da yanıtlanmalıdır. Bu çalışma, geometriyi aşan dört genişlemeyi sistem düşüncesi geleneğinden faydalanarak ele almaktadır.

## Hiyerarşik Çerçeve: Yapıdan Paradigmaya

Söz konusu katmanlı çerçeveleme, sistem düşüncesi literatüründe Donella Meadows tarafından 1999 yılında yayımlanan *Leverage Points: Places to Intervene in a System* adlı çalışmada sistemleştirilmiştir. Meadows bir sistemin değişim için müdahale noktalarını sırasıyla parametreler, stoklar, akışlar, geri besleme döngüleri, kurallar, hedefler ve paradigmalar olarak sıralar. Üst sıralardaki katmanlar daha yüksek kaldıraç taşıırken aynı zamanda daha güç değiştirilebilir nitelik göstermektedir. Yazılım mimarisi bağlamına uyarlandığında bu hiyerarşi şu biçim almaktadır:

1. **Yapı (geometri):** modüller, bağımlılık kenarları, kütle dağılımı.
2. **Dinamik:** yapının zaman içindeki değişimi ve değişim örüntüleri.
3. **Kuvvet:** bu değişimi üreten organizasyonel ve süreç düzeyindeki nedenler.
4. **İşlev:** yapının ürettiği davranışsal sonuçlar.
5. **Niyet:** sistemin hangi amaca hizmet etmek üzere kurgulandığı.
6. **Anlam:** sistemin insan deneyimine ne kattığı.

Geometri yalnızca bu hiyerarşinin ilk katmanını temsil etmektedir. Geri kalan beş katman, geometriyi üreten ve onun yorumlanmasını mümkün kılan bağlamı oluşturur.

## Birinci Genişleme: Dinamik (Behavioral Code Analysis)

Geometrinin durağan bir fotoğraf olduğu yerde, dinamik aynı sistemin zaman içindeki hareketini tarif etmektedir. Bu alan yazılım mühendisliği literatüründe Adam Tornhill tarafından *behavioral code analysis* adıyla sistemleştirilmiş ve iki temel eserle ortaya konulmuştur: *Your Code as a Crime Scene* (2015) ve *Software Design X-Rays* (2018). Yöntemin temel veri kaynağı dosya sisteminin durağan görüntüsü değil, sürüm kontrol geçmişi.

Bu yaklaşımda üç temel metrik özellikle belirleyicidir:

- **Hotspot:** Belirli bir zaman aralığında en sık değişen dosyalar. Sürekli değişen bir dosya genellikle ya yoğun gelişim altındadır ya da mimari olarak yerleşmemiştir; her iki durumda da yüksek risk taşımaktadır.
- **Geçici eşleşme (temporal coupling):** Aralarında doğrudan içe aktarma ilişkisi bulunmadığı halde sürekli birlikte değiştirilen dosyalar. Bu örüntü, statik bağımlılık çözümleyicilerin yakalayamadığı gizli bir mimari bağı işaret eder.
- **Bilgi yoğunluğu ve truck factor:** Belirli bir modülün geliştirme tarihinin kaç farklı katılımcıya dağıldığı. Tek bir kişinin hâkim olduğu bir modül, o kişinin sistemden ayrılması durumunda kritik bir bilgi boşluğu üretmektedir.

Bu metriklerin temel düzeyde elle hesaplanması için git geçmişi yeterli olmaktadır. Örneğin son altı aydaki hotspot dağılımı şu komutla elde edilebilir:

```
git log --since=6.months --format=format: --name-only \  
| grep -v '^$' \  
| sort | uniq -c | sort -rn | head -20
```

Daha kapsamlı analizler için CodeScene aracı (Tornhill'in geliştirdiği ticari ve açık-kaynak sürümleri bulunan platform) tüm bu metrikleri ve geçici eşleşme grafiğini görselleştirilmiş biçimde sunmaktadır.

Dinamik analizin geometriye eklediği temel içgörü şudur: bir dosyanın büyük olması tek başına bir risk göstergesi değildir; ancak büyük ve sürekli değişen bir dosya, sürdürülebilirlik açısından kritik bir basınç noktasıdır. Geometri yalnızca büyüklüğü; dinamik ise bu büyüklüğün hangi hızda ve hangi yönde evrildiğini ölçer.

## İkinci Genişleme: Kuvvet (Conway’s Law)

Bir kod tabanının geometrisi tesadüfen ortaya çıkmamaktadır; onu üreten yapısal kuvvetler vardır. Bu kuvvetlerin en temel ifadesi, Melvin Conway tarafından 1967 yılında formüle edilmiş ve bugün *Conway’s Law* olarak anılan tezdır: “*Bir sistemi tasarlayan organizasyonlar, kendi iletişim yapılarının kopyası olan tasarımlar üretmek zorundadır.*”

Bu yasaya göre kod tabanının topolojisi, onu üreten takımın iletişim topolojisine yapısal olarak izomorftur. Kardeş bağı çoğu zaman aynı modüller üzerinde çalışan iki geliştiricinin doğrudan iletişiminden doğmaktadır; aşırı merkezileşmiş bir tip dosyası, organizasyonda herkesin başvurduğu merkezi bir kişinin kristalleşmiş yansıması olabilmektedir; sınır katmanındaki çift yönlü baskı ise frontend ve backend takımlarının tek bir kişi üzerinden iletişim kurduğu durumların ürünüdür.

Bu gözlemin pratik sonucu, *inverse Conway maneuver* olarak adlandırılan yaklaşımda formüle edilmiştir: hedef mimariye ulaşmak isteniyorsa, organizasyon yapısı hedef mimariyi yansıtacak biçimde yeniden düzenlenmelidir. Aksi takdirde geometrik düzeltmeler kalıcı olmamakta; sistem zaman içinde kendisini üreten organizasyonel topolojinin biçimine geri dönmektedir.

Kuvvetler tek başına organizasyon yapısıyla sınırlı değildir. Pratikte sıkça gözlenen bir geri besleme döngüsü şu biçimde çalışmaktadır: zaman baskısı kısa yol kararı üretir; kısa yol kardeş bağı üretir; kardeş bağı yapısal yumak üretir; yumak yeniden yapılandırma maliyetini artırır; yüksek yeniden yapılandırma maliyeti yeni kısa yol kararlarını teşvik eder. Geometri bu döngünün yalnızca bir kesitini gösterir; kuvvet düzeyinde müdahale edilmediği takdirde aynı geometri tekrar üretilmektedir.

## Üçüncü Genişleme: Niyet

Bir mimarinin sağlığı, kendiliğinden değil; hizmet ettiği niyete göre değerlendirilmelidir. “Bu mimari iyi midir?” sorusu yanıt vermeye uygun değildir; çünkü iyilik bağlama göreler. Doğru formülasyon şu biçimdedir: *Bu mimari, sistemin niyetine uygun mudur?*

Aynı geometrik yapı, farklı niyetler altında farklı değerlendirmelere konu olmaktadır. Hızlı doğrulama hedefli bir prototip için kabul edilebilir olan bir gravity well, uzun ömürlü bir platformda kritik bir zaaf olabilmektedir. Regülasyon altındaki bir finansal sistemde zorunlu olan iz kayıt katmanı, bir demo uygulamasında gereksiz bir hacim olarak görünebilmektedir. Fitness function eşik değerlerinin evrensel olmamasının nedeni de aynıdır: eşikler niyetin türetilmiş sonuçlarıdır.

Niyetin yazılı olarak kayıt altına alınması için yerleşik bir araç Mimari Karar Kayıtlarıdır (Architecture Decision Records, ADR). Michael Nygard tarafından 2011 yılında önerilen bu yapı, her mimari kararı şu alanlarla belgeler: bağlam (hangi koşullar altında karar verildi), karar (neye karar verildi), sonuçlar (kararın olumlu ve olumsuz tarafları). ADR’ler niyeti zaman içinde izlenebilir kılar ve

sonraki yeniden yapılandırma kararlarının yalnızca geometriye değil; geometriyi şekillendiren niyete de uygun verilmesini sağlar.

## Dördüncü Genişleme: Anlam

Hiyerarşinin son katmanı, sistemin yapısının insan deneyimine nasıl çevrildiğidir. Bu katman mühendislik literatürünün dışında, mimari teorisinde Christopher Alexander tarafından sistemleştirilmiştir. Alexander'ın *The Timeless Way of Building* (1979) ve *A Pattern Language* (1977) eserlerinde merkezi kavram *quality without a name* olarak ifade edilen, formun arkasındaki yaşayan niteliktir. Alexander'a göre iyi bir form yalnızca yapısal olarak doğru değil; aynı zamanda kullanıcının yaşantısında belirli bir bütünlük üretendir.

Yazılım bağlamında bu katmanın somut karşılığı şu sorulardır: Sistem hangi karmaşıklığı kullanıcıdan gizleyebilmektedir? Hangi iş akışını sadeleştirmektedir? Hangi yanlış adımı imkânsız kılmaktadır? Bu sorular geometriyle doğrudan bağlantılıdır; çünkü iyi yerleştirilmiş bir sınır, kullanıcı arayüzünde de bir netlik üretmektedir. Buna karşılık karmaşık ve yumak halindeki bir geometri, çoğu zaman karmaşık ve hata üreten bir kullanıcı deneyimine yansımaktadır.

Bu katmanın doğrudan ölçülemiyor olması, önemsiz olduğu anlamına gelmemektedir; yalnızca operasyonel müdahalenin diğer katmanlara göre daha dolaylı olduğunu göstermektedir.

## Operasyonel Yansıma: Hangi Soru Hangi Katmanda

Bir mimari soruyla karşılaşıldığında ilk yapılması gereken, sorunun hangi katmana ait olduğunun saptanmasıdır. Her katmanın kendine özgü ölçüm araçları ve müdahale yolları bulunmaktadır:

- Geometri katmanı: bağımlılık çözümleyicileri, içe aktarma sayımı, dosya hacmi dağılımı, force-directed graph görselleştirmeleri.
- Dinamik katmanı: git geçmişi analizi, hotspot raporları, geçici eşleşme matrisi, truck factor hesaplaması, CodeScene benzeri araçlar.
- Kuvvet katmanı: organizasyon şeması analizi, takım iletişim haritaları, geliştirme süreç gözden geçirmesi, Conway maneuver planlaması.
- Niyet katmanı: ADR'ler, ürün stratejisi belgeleri, mimari karar kayıtları, paydaş görüşmeleri.
- Anlam katmanı: kullanıcı araştırması, kullanılabilirlik testleri, kullanıcı deneyimi gözlemi, ürün geri bildirimi.

Yanlış katmanda yapılan müdahale, doğru katmanda kalan problemi çözmemektedir. Örneğin organizasyonel kuvvetler sürdükçe yapılan geometri refactor'u kalıcı olmaz; net bir niyet kaydı bulunmayan bir sistemde belirlenen fitness function eşikleri keyfi kalmaktadır.

## Sınırlama: Hiyerarşinin Üst Katmanları Daha Az Operasyoneldir

Bu altı katmanlı hiyerarşi her ne kadar tutarlı bir analitik çerçeve sunsa da, katmanlar arasında operasyonel uzaklık açısından belirgin bir asimetri bulunmaktadır. Geometri katmanında “merkezileşmiş tip dosyasını paylaşımlı ad alanına taşı” biçiminde somut bir eylem tanımlanabilirken, anlam katmanında “sistem kullanıcının zihinsel yükünü hafifletmelidir” biçiminde yalnızca yönlendirici bir ilke söylenebilmektedir.

Bu nedenle hiyerarşinin yukarı kısımlarına çıkmak yalnızca bağlamı zenginleştirme amacıyla yapılmalı; alt katmanlardaki somut eylemlerin yerine geçirilmemelidir. Mimari pratikte gözlenen tipik bir başarısızlık örüntüsü, üst katmanların felsefi olarak işlenip alt katmanlarda somut bir adım atılmamasıdır. Tersine de geçerlidir: üst katmanlar dikkate alınmadan yürütülen sürekli geometri müdahaleleri, kuvvet kaynağını ele almadıkları için tekrar eden çevrimler üretmektedir.

## Sonuç

Doğru yaklaşım, müdahaleyi en somut katmandan (geometri) başlatmak, ancak bu müdahalenin gerekçelendirmesini ve sürdürülebilirliğini üst katmanlardan (dinamik, kuvvet, niyet) türetmektir. Geometri sistemin şeklini verir; üst katmanlar bu şeklin neden böyle olduğunu, ne yöne evrildiğini ve neye hizmet ettiğini açıklar. İkisinin birlikte ele alındığı bir analiz, bir kerelik müdahale yerine sürdürülebilir bir mimari pratiğin temelini oluşturmaktadır.